

Lógica e Inteligencia Artificial -

Práctica 9 - Machine Learning y

Redes Neuronales Artificiales

Temario

- **Inteligencia artificial no-simbólica: Algoritmos de aprendizaje automático (Machine learning), Redes neuronales artificiales. Algoritmos de entrenamiento de redes neuronales artificiales. Backpropagation.**

Referencias

- **Videos del curso Nociones de Deep Learning para Inteligencia Artificial Generativa de Texto**
 - [Como programar un Autograd desde cero - Clase 1](#)
 - [Como programar un Autograd desde cero - Clase 2](#)
 - [Haciendo Backpropagation a mano - Clase 3](#)
 - [Extendiendo la clase Value - Clase 4](#)
 - [MyNN, implementando tu propia librería de Redes Neuronales - Clase 1](#)
- **Videos de Andrej Karpathy sobre Deep Learning**
 - [Neural Networks: Zero to Hero by Andrej Karpathy](#)
 - **Video 1 - building micrograd**
 - **Video 5 - makemore part4, becoming backprop ninja**
- **Understanding Deep Learning by Simon J. D. Prince**
 - [Understanding Deep Learning](#)
 - **Chapter 3 - Shallow Neural Networks**
 - **Chapter 4 - Deep Neural Networks**
 - **Chapter 5 - Loss Functions**
 - **Chapter 6 - Fitting Models**
 - **Chapter 7 - Gradient and Initialization (incluye la explicación del algoritmo de Backpropagation).**
- [Activation Functions Explained](#)

Fecha de entrega

15 de diciembre de 2025.

Autor

- **Nombre: Juan Cruz**
- **Apellido: Cassera Botta**
- **Legajo: 17072/7**

Introducción

Michigrad es un motor de cálculo automático de gradientes para valores escalares diseñado con fines pedagógicos. Funciona con los mismos principios matemáticos que PyTorch o Tensorflow, construyendo el grafo de operaciones de la función de costo y permitiendo calcular el gradiente parcial para cada nodo del grafo, lo que permite aplicar el algoritmo de Backpropagation.

Cada nodo del grafo es un elemento de la clase `Value` definida en el archivo `michigrad/engine.py`. Las operaciones sobre `Values` que están implementadas permiten sumar (`__add__`), multiplicar (`__mul__`), exponenciar (`exp`), y calcular la potencia a un número entero (`__pow__`). Además está implementada la función de activación `relu`.

Cada operación sabe como se calcula el gradiente local (en la función local `_backward`), calculando la derivada de la operación. Para la suma, la derivada es constante con lo que solamente se propaga el gradiente. Para la multiplicación, se propaga el gradiente multiplicado por la derivada parcial respecto a cada una de las variables y viceversa. Si tengo -5×6 , la derivada con respecto a 6 es -5 y con respecto a -5 es 6.

Además, la clase `Value` implementa la función `backward()` que permite calcular todos los gradientes desde el nodo raíz (generalmente el resultado de la función de pérdida) usando un sort topológico del grafo de operaciones.

Descripción del trabajo

[Hacer un fork y clonar el repositorio de código de Michigrad](#) visto en clase.

Parte 1

Usando la librería MyNN, implementar una red neuronal que prediga la salida de la función XOR. Usar una única capa con dos neuronas lineales (sin ReLU). Graficar el grafo de operaciones de una secuencia de entrenamiento, luego de la etapa de forward y al finalizar la primera pasada de backpropagation.

Iterar varias veces el bucle de entrenamiento:

1. Forward pass
2. Loss
3. Zero grad
4. Backward pass
5. Update

Compruebe que la función de pérdida disminuye en cada iteración, verificando que el modelo está aprendiendo.

Realizar una pequeña explicación de entre 100 y 200 palabras sobre por qué cree que el modelo es incapaz de representar la función correctamente. Si bien esta respuesta es muy personal, puede consultar con sus compañeros antes de elaborar la respuesta. Evite usar un modelo de lenguajes, salvo para la edición.

NOTA: El repo forkeado para realizar el trabajo se encuentra en este [link](#).

Planteo del problema:

Dado el operador lógico XOR, se quiere construir una red neuronal de una sola capa con dos neuronas lineales, sin ReLU, que prediga la salida de este operador. La tabla del XOR es la siguiente:

0	0	0
0	1	1
1	0	1
1	1	0

Explicación de la solución (archivos Parte1.ipynb, nn.py, engine.py):

La solución se implementa en un archivo Jupyter Notebook con el nombre indicado. En la primera etapa se preparan las variables que se necesitan para el entrenamiento:

1. Se importan las clases `Value` y `Layer`, además de la función `show_graph()`.
2. Se definen todas las entradas y las salidas correspondientes a esas entradas, según la tabla XOR descrita arriba.
3. Se define la capa con 2 neuronas, 2 entradas por neurona (cada fila de la tabla tiene dos entradas y una salida), y sin ReLU.

En la segunda etapa se implementa el entrenamiento de la red:

1. Se define un learning rate de 0.05. Este valor controla qué tanto se ajustan los pesos del modelo durante el entrenamiento, en cada pasada de backpropagation.
2. Se define el bucle de entrenamiento con un for.
3. Se crea un objeto `Value` con valor inicial 0 que representa a la función de pérdida L.
4. Se realiza el Forward Pass y el cálculo de la pérdida:
 - a. Se iteran las entradas y las salidas, donde para cada terna de dos valores de entrada y un valor de salida, se evalúa el resultado de la red con esas dos entradas, que en realidad son dos resultados porque tenemos dos neuronas.
 - b. Se suman los dos resultados de estas dos neuronas.
 - c. Se calcula la pérdida, es decir se actualiza el valor del objeto `Value` que representa la función de pérdida acumulándole el error: la salida esperada - la suma anterior al cuadrado.
5. Se reinician los gradientes de todos los parámetros a cero antes de hacer Backpropagation.
6. Se realiza Backpropagation:
 - a. Empezando desde el nodo definido en el paso 3, se calculan todos los gradientes de los nodos en orden topológico inverso.
7. Se actualizan los pesos de los parámetros.

Gráfico del grafo de operaciones en la primer secuencia de entrenamiento, luego de la etapa de forward:

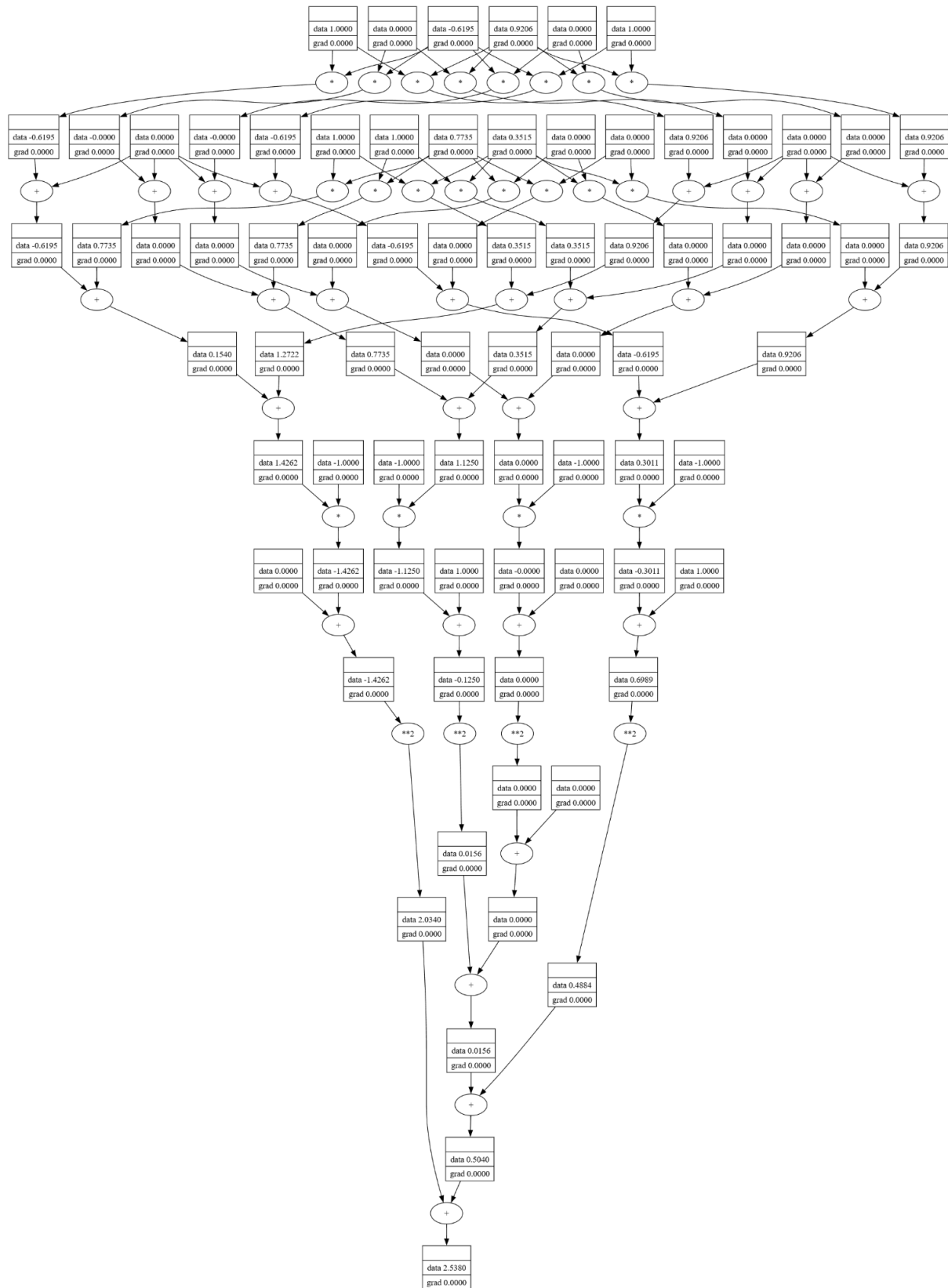
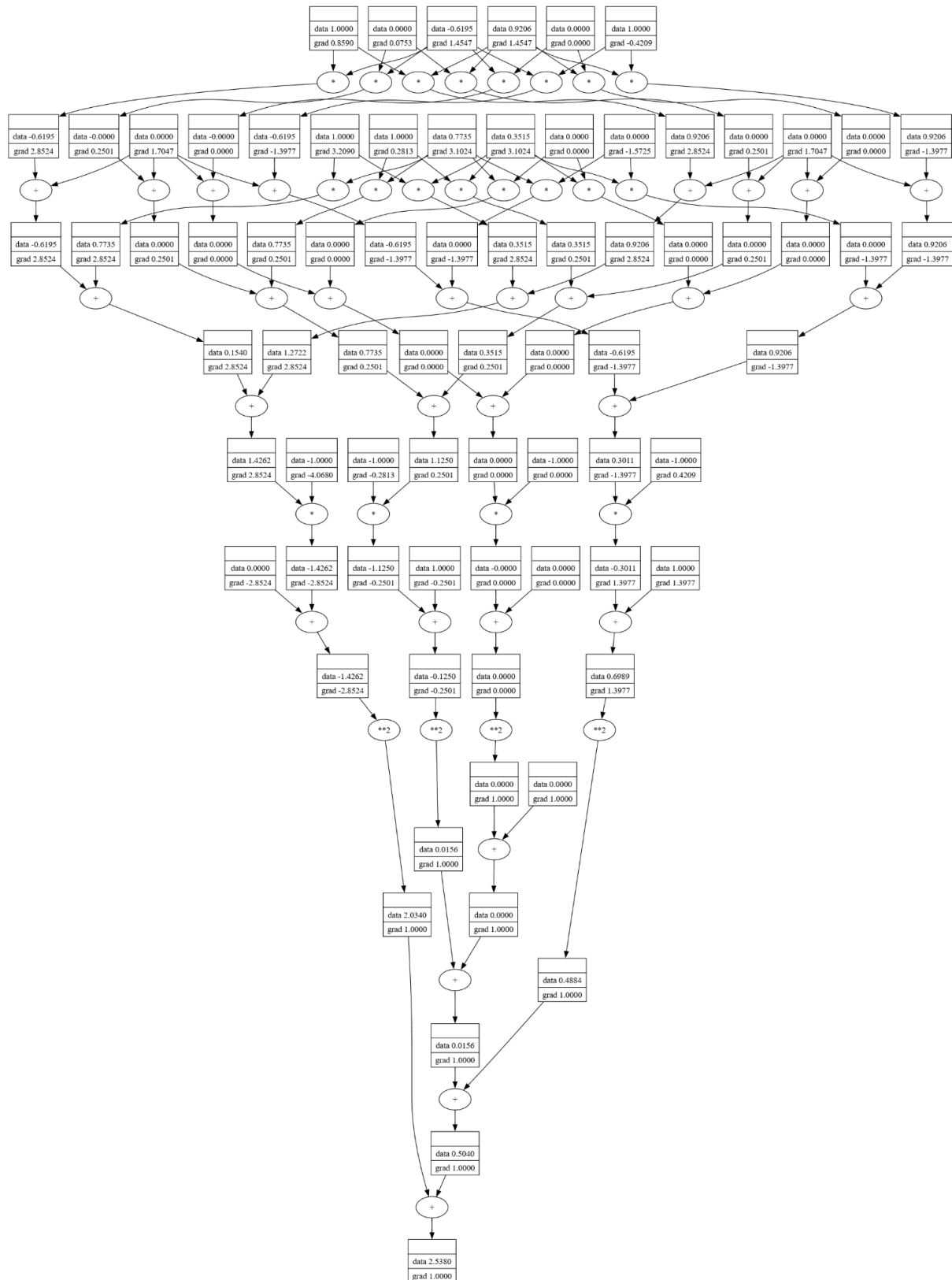


Gráfico del grafo de operaciones en la primer secuencia de entrenamiento, luego de la primer pasada de Backpropagation:

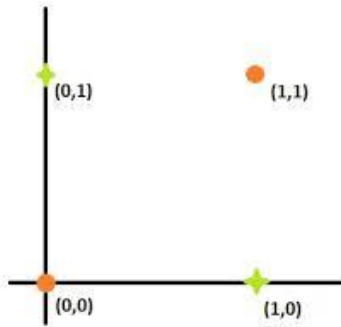


Comprobando que la función de pérdida disminuye en cada iteración:

Iteración 0: pérdida = 2.5380304707932613
Iteración 2: pérdida = 1.5808302799217677
Iteración 4: pérdida = 1.3138111673791983
Iteración 6: pérdida = 1.1735132933328782
Iteración 8: pérdida = 1.0973731883393167
Iteración 10: pérdida = 1.0552636230340575
Iteración 12: pérdida = 1.031628811024793
Iteración 14: pérdida = 1.0182135027300188
Iteración 16: pérdida = 1.0105349396230465
Iteración 18: pérdida = 1.00611296168314
Iteración 20: pérdida = 1.0035551189097072
Iteración 22: pérdida = 1.0020708697536826
Iteración 24: pérdida = 1.0012076552322544
Iteración 26: pérdida = 1.000704821868792
Iteración 28: pérdida = 1.0004115847200503
Iteración 30: pérdida = 1.0002404418886397
Iteración 32: pérdida = 1.0001405015263396
Iteración 34: pérdida = 1.0000821175801897
Iteración 36: pérdida = 1.0000480009987998
Iteración 38: pérdida = 1.0000280611705148
Iteración 40: pérdida = 1.0000164055309182
Iteración 42: pérdida = 1.0000095916896088
Iteración 44: pérdida = 1.0000056080797421
Iteración 46: pérdida = 1.0000032790136084
Iteración 48: pérdida = 1.0000019172522046

Sin embargo, se puede ver claramente que si bien el modelo está aprendiendo, su error nunca se acerca a 0, si no que se estanca acercándose infinitamente a 1. Esto indica que el modelo es incapaz de representar correctamente la función XOR.

La razón por la cual el modelo no puede representar correctamente el XOR tiene que ver con el gráfico de la tabla de esta función:



Se puede ver que las entradas $(0,0)$ y $(1,1)$ que dan como resultado 0 están en esquinas exactamente opuestas, y lo mismo ocurre con las entradas $(0,1)$ y $(1,0)$ que dan como resultado 1, también están las otras dos esquinas opuestas.

Lo que una red neuronal 100% lineal, sin ReLU (como la que se implementó) hace es intentar encontrar una línea recta que separe claramente a todas las entradas que producen una determinada salida de las entradas que producen otra salida distinta. En este caso, la RN intentaría trazar una línea que deje a todos los puntos naranjas de un lado, y a los verdes del otro. Pero esto es imposible en este caso, porque sin importar de qué forma trace la línea recta, siempre dejará a algún punto del lado erróneo.

La solución a este problema sería añadir no linealidad al modelo, como por ejemplo utilizando la función de activación ReLU.

Parte 2

Extender la clase **Value** para que soporte otras funciones de activación además de ReLU. Implementar la función Tanh y la función Logística (sigmoide). Aquí la dificultad radica en calcular las derivadas parciales de las funciones y en cómo propagarlas a los nodos superiores.

Para Tanh por ejemplo, la función está definida como:

$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

y su derivada es:

$$\tanh'(x) = 1 - \tanh(x)^2 = 1 - \left(\frac{e^{2x} - 1}{e^{2x} + 1} \right)^2$$

La función Logística está definida como:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

y su derivada es:

$$\text{sigmoid}'(x) = \frac{e^x}{(1 + e^{-x})^2}$$

Modificar **nn.py** para que ReLU, Tanh y Sigmoid se comporten como capas en lugar de ser un parámetro de **Neuron**.

Extendiendo la clase **Value (archivo `engine_extended.py`):**

Para que la clase **Value** soporte las funciones de activación Tanh (tangente hiperbólica) y Sigmoide (función logística), se añaden dos métodos a esta clase:

1. Método **tanh(self)**:
 - a. Este método tiene una estructura muy similar a la del método **relu(self)** donde primero se calcula el nuevo objeto **Value** que se obtiene a partir de aplicar la función tangente hiperbólica al objeto **Value** actual (**self**), y luego se define cómo será el backward pass

de este nuevo objeto, usando la derivada de la función tangente hiperbólica.

- b. Para calcular el dato del nuevo objeto **Value** a crear, se calcula la función $\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$ donde en este caso x es el dato del objeto actual, es decir **self.data**. Luego, este nuevo objeto no tiene hijos salvo a sí mismo, y el nombre de la operación que lo creó es “tanh”.
- c. Finalmente se le agrega la función **_backward()** a su atributo **_backward**. Esta función acumula el gradiente del objeto **Value** actual como la derivada de tanh, que es $\tanh'(x) = 1 - \tanh(x)^2$ multiplicada por el gradiente del nuevo objeto **Value**, por la regla de la cadena.

2. Método **sigmoid(self)**:

- a. Nuevamente la estructura es la misma, se calcula el dato del objeto nuevo, y se le define su backward pass.
- b. El dato del objeto nuevo se obtiene con la definición de la función sigmoide: $\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$ donde x es nuevamente **self.data**. Como antes, el nuevo objeto no tiene hijos salvo a sí mismo, y el nombre de la operación es “sigmoid”.
- c. Para el backward pass, se define la función **_backward()** que acumula el gradiente del objeto **Value** actual como la derivada de sigmoide: $\text{sigmoid}'(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$ multiplicada por el gradiente del nuevo objeto **Value**, por la regla de la cadena.

Modificando nn.py (archivo nn_refactored.py):

Hecho lo anterior, ahora se quiere que ReLU, Tanh y Sigmoid se comporten como capas en lugar de ser un parámetro de **Neuron**.

Para esto se crean 3 clases nuevas dentro de **nn.py**, una para ReLU, otra para Tanh, y otra para Sigmoid, todas heredan de **Module**.

Lo único que definen estas clases es el método **__call__**, que recibe una lista de objetos **Value**, itera sobre ellos, y llama al método correspondiente para cada uno de ellos. En la clase ReLU, cada objeto **Value** que recibe llama al método **obj.relu()**, en la clase Tanh cada objeto **Value** que recibe llama al método **obj.tanh()**, y así siguiendo. Luego, devuelve el o los resultados de estas llamadas, dependiendo de cuántos objetos recibe, si uno o varios.

A su vez, dentro de la clase **Neuron** se hacen los siguientes cambios:

1. Se elimina el argumento **nonlin** en el constructor.
2. Se elimina el atributo **self.nonlin**.
3. Se modifica el método **__call__** para que siempre devuelva la suma ponderada + el sesgo de forma totalmente lineal siempre.

Dentro de la clase **Layer** se elimina el argumento ****kwargs** ya que la clase **Neuron** ya no lo necesita.

Parte 3

Rehaga el ejercicio de la función XOR usando ahora una capa de dos neuronas con alguna función de activación (ReLU, Tanh o Sigmoid).

Realizar una pequeña explicación de entre 100 y 200 palabras donde interpreta los resultados del modelo entrenado. Como en el caso anterior, si bien esta respuesta es muy personal, puede consultar con sus compañeros antes de elaborar la respuesta. Evite usar un modelo de lenguajes, salvo para la edición.

Explicación de la solución (archivos `Parte3.ipynb`, `engine_extended.py`, `nn_refactored.py`):

Al igual que en la parte 1, la solución se implementa en un archivo Jupyter Notebook con el nombre indicado.

La primera etapa es igual que antes, solo que ahora no pasamos el argumento `nonlin` a la clase `Layer` ya que no lo tiene más, y además creamos un objeto `activacion` que es de la clase `Tanh`.

La segunda etapa es muy similar a la parte 1, solo que dentro del paso de forward pass, primero obtenemos la salida predicha por la red y antes de compararla con la salida esperada, le aplicamos la función tangente hiperbólica usando el objeto `activacion` a esas salidas predichas.

Gráfico del grafo de operaciones en la primer secuencia de entrenamiento, luego de la etapa de forward:

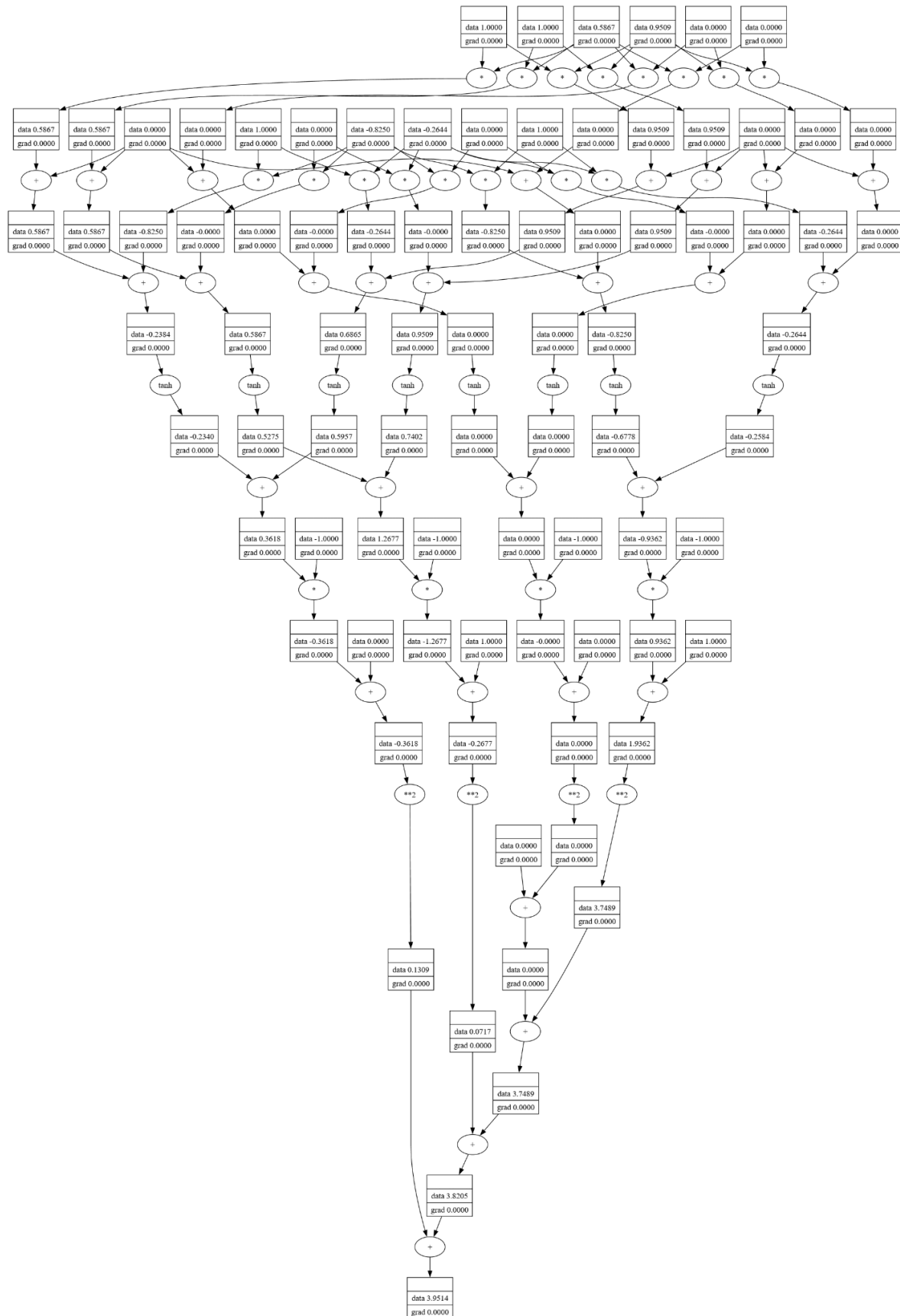
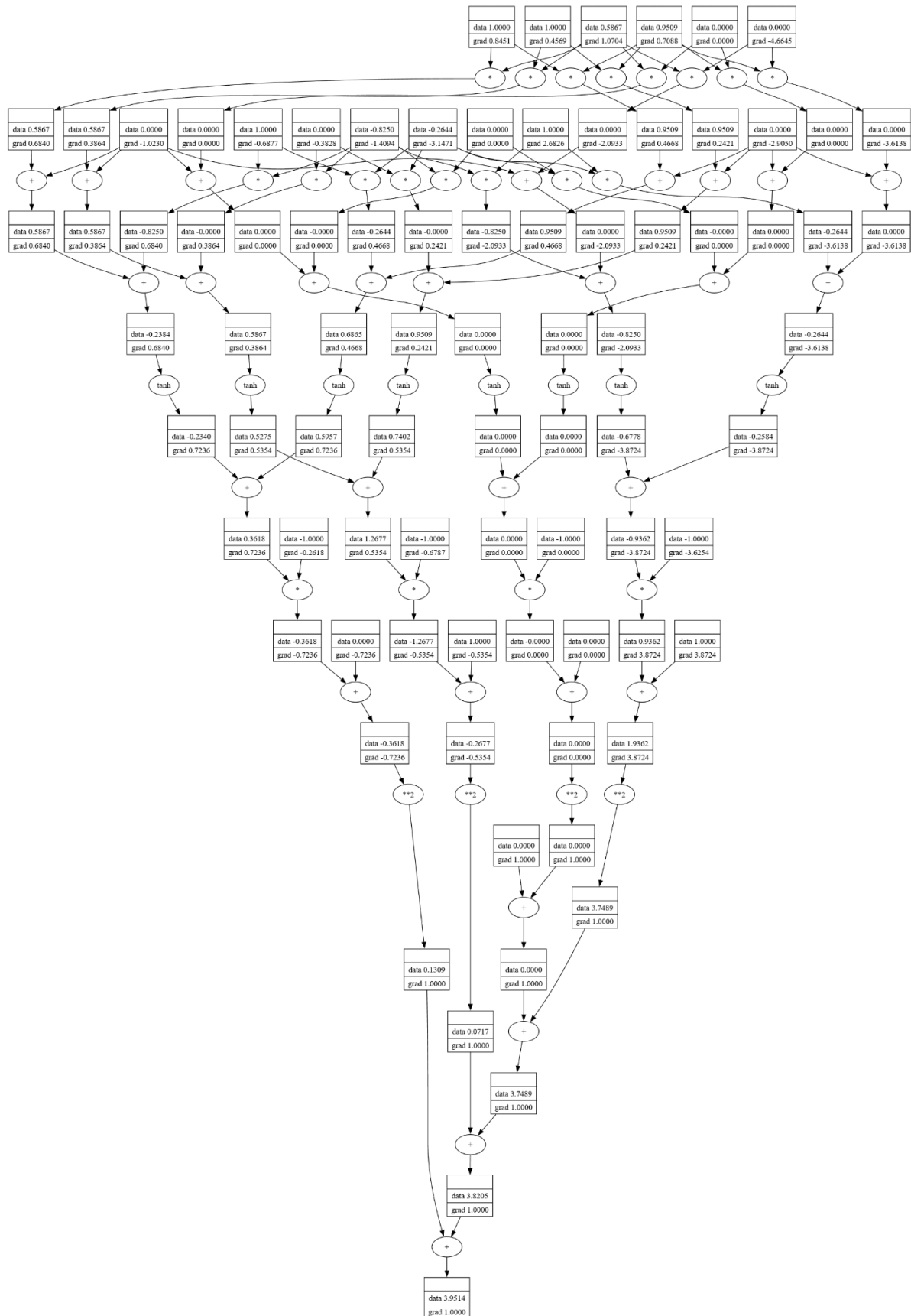


Gráfico del grafo de operaciones en la primer secuencia de entrenamiento, luego de la primer pasada de Backpropagation:

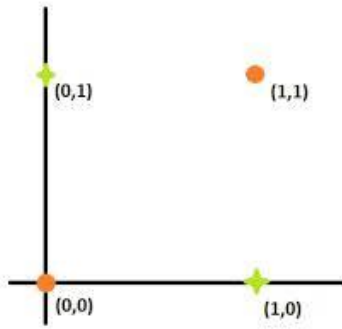


Comprobando que la función de pérdida disminuye en cada iteración:

Iteración 0: pérdida = 3.95141791380134
Iteración 2: pérdida = 2.2702947762268715
Iteración 4: pérdida = 1.5662221009411592
Iteración 6: pérdida = 1.2467294905676214
Iteración 8: pérdida = 1.0855869013512176
Iteración 10: pérdida = 0.9906211943734499
Iteración 12: pérdida = 0.9272945344335337
Iteración 14: pérdida = 0.8814803945652506
Iteración 16: pérdida = 0.846521075568299
Iteración 18: pérdida = 0.818810942628913
Iteración 20: pérdida = 0.7961571148305822
Iteración 22: pérdida = 0.7771093775336139
Iteración 24: pérdida = 0.7606572283746378
Iteración 26: pérdida = 0.746076628856724
Iteración 28: pérdida = 0.7328426023036256
Iteración 30: pérdida = 0.7205736486233328
Iteración 32: pérdida = 0.7089934462680572
Iteración 34: pérdida = 0.6979031775199435
Iteración 36: pérdida = 0.6871610726177207
Iteración 38: pérdida = 0.6766671800961371
Iteración 40: pérdida = 0.6663520382670607
Iteración 42: pérdida = 0.6561682866836558
Iteración 44: pérdida = 0.6460844899126033
Iteración 46: pérdida = 0.636080615564727
Iteración 48: pérdida = 0.6261447397594652

Esta vez la función de pérdida ya no se estanca en 1 como antes, si no que va acercándose a cero de a poco (aún está lejos de cero por haber hecho pocas iteraciones, pero si se hacen más iteraciones nos acercaremos a un error más y más cercano a cero), lo que implica que la red no solo está aprendiendo, si no que mejora continuamente y eventualmente será capaz de predecir con mucha exactitud la tabla de verdad del XOR.

Lo anterior claramente se debe a haber agregado no linealidad a la red, mediante la tangente hiperbólica. Esta función es no-lineal y si la vemos en un gráfico es una curva. Si analizamos el gráfico del XOR nuevamente:



Podemos ver que si bien la separación es imposible con una línea recta (red neuronal 100% lineal), sí es posible hacerlo con una curva, que es exactamente lo que tanh hace.